

Received 18 February 2025, accepted 8 March 2025, date of publication 12 March 2025, date of current version 20 March 2025.

Digital Object Identifier 10.1109/ACCESS.2025.3550665

RESEARCH ARTICLE

Integrating Time Series Anomaly Detection Into DevOps Workflows

GUSTAV KÅNÅHOLS¹, SHAHRIAR HASAN¹, AND PER ERIK STRANDBERG²

¹School of Innovation, Design and Engineering, Mälardalen University, 721 23 Västerås, Sweden

²Westermo Network Technologies AB, 721 30 Västerås, Sweden

Corresponding author: Shahriar Hasan (shahriar.hasan@mdu.se)

This work was supported in part by the ECSEL Joint Undertaking (JU) funded by the European Union's Horizon 2020 Research and Innovation Program and Sweden, Austria, Czech Republic, Finland, France, Italy, and Spain, under Grant 101007350; and in part by the Swedish Knowledge Foundation (KKS) TeleDrive under Project 20230076.

ABSTRACT Anomaly detection in the monitoring systems of DevOps environments is crucial for ensuring system reliability, preventing downtime, and maintaining the efficiency of continuous integration and continuous deployment pipelines. Artificial Intelligence (AI)-based solutions for automated anomaly detection in DevOps workflows are attracting growing research interest. However, challenges remain regarding the lack of ground truth data from DevOps systems, as well as difficulties in storing, processing, and visualizing the collected data. Furthermore, most of these datasets are unlabeled, making it unclear what constitutes anomalous system behavior, and no generalized approach exists for selecting the most suitable AI algorithms for anomaly detection in such contexts. To address these challenges, this paper publishes a comprehensive time series dataset from 19 different DevOps test systems, comprising 24 performance metrics sampled twice per minute over 30 days. Moreover, to benchmark the dataset, we first label a subset of the dataset based on feedback from the DevOps experts within the industry context. Then six different algorithms are employed on the dataset and their performance in anomaly detection is evaluated using three different Area Under the Curve (AUC) metrics. Additionally, this paper presents a tool for storing, visualizing, monitoring, and integrating anomaly detection algorithms and makes it available to the community. The performance evaluation results aiming to establish a baseline for further research show that AI-based anomaly detection can significantly benefit DevOps workflows but emphasize the need for algorithm selection and parameter tuning tailored to the specific industry context and dataset.

INDEX TERMS Anomaly detection, deep learning, DevOps, machine learning, time series analysis.

I. INTRODUCTION

DevOps is a methodology that combines aspects of agile software development (Dev) and operations (Ops) to increase the speed and quality at which software is delivered [1]. An important part of DevOps is the use of software testing to ensure the quality of the software. This is done by running automated tests on the software to detect potential issues. The tests are often run in a continuous integration/continuous deployment (CI/CD) pipeline, where the software is built, tested, and deployed automatically. When performing such tests, the systems running them might experience issues and

enter abnormal states. Various factors, such as hardware failures, software bugs, or network issues can cause these issues. The results of tests running on systems in abnormal states might be unreliable and can affect the quality of the testing. Anomaly detection is the process of identifying such abnormal states [2] and is an important part of ensuring the validity of the tests.

Research into anomaly detection has spanned over two centuries, covering multiple domains with applications ranging from detecting election fraud to identifying faults in manufacturing equipment [3]. Detection of anomalies can be performed on various types of data, such as images, text, or time series, depending on the domain. A *time series* is a sequence of data points ordered by time measuring the change

The associate editor coordinating the review of this manuscript and approving it for publication was Porfirio Tramontana¹.

of some variable over time [4], such as the heart's electrical activity in an electrocardiogram, the power demand in the electrical grid, or the temperature of a server in a data center. Anomalies are deviations from the expected behavior of a time series in some form. The anomalies can be single data points or a sequence of data points and can be caused by a variety of reasons depending on the application [5]. For example, a spike in a server's temperature might indicate a hardware failure or software bug, while a flat line in the power demand of the electrical grid might indicate a power outage.

To detect anomalies, it is essential to have a clear understanding of what anomalies can be expected to occur in the data. As a baseline, the work presented in this paper considers the ground truth time series data obtained from the test systems of Westermo Network Technologies AB (Westermo), which develops hardware (HW) and software (SW) for cyber-physical systems (CPS) such as switches and routers used in industrial networks for on-board rail, power distribution, mines, and other industrial automation sectors. As part of their testing infrastructure, the Westermo test systems generate a wide range of performance metrics, such as CPU usage, memory usage, and network traffic, which are employed to monitor system health. Without loss of generality, software companies following DevOps workflows face challenges in managing monitoring data, including its storage, processing, and visualization [6], [7], [8]. Furthermore, timely detection of anomalies in such monitoring systems enables DevOps teams to operate more efficiently and prevent system failures, which could otherwise lead to severe consequences. Especially, due to the increasing complexity of software systems, generating high volume and complex data, small faults in the system can potentially lead to domino effects and a cascade of failures [9]. The traditional anomaly detection approaches in DevOps environments struggle to identify meaningful patterns from such complex and large volumes of data.

When addressing anomaly detection in time series data, the lack of labeled anomalous data [10] presents a significant challenge in distinguishing between normal and anomalous data points within the time series. The problem is further compounded by the variability in DevOps workflows and limited generalizability in both data and anomaly detection methods. As a result, it becomes essential to first define what constitutes an anomaly within the context of a specific dataset and its corresponding application domain. Additionally, evaluating anomaly detection algorithms based solely on the anomalies present in a given dataset may be insufficient, as the dataset could be contaminated with undetected anomalies. Such contamination could undermine its reliability as training data for learning-based algorithms [11].

An intelligent anomaly detection system is clearly essential to ensure the quality of tests conducted on test systems. Existing research demonstrates that both traditional statistical methods and Machine Learning (ML) algorithms, including supervised and unsupervised approaches, are effective for

anomaly detection [12]. However, a one-size-fits-all solution does not exist; the suitability of specific algorithms depends on the context and the types of anomalies present in a given dataset. Consequently, evaluating the performance of various learning models in detecting anomalous behaviors within the context of a specific dataset becomes imperative. Moreover, it is crucial to determine the performance metrics suitable for carrying out such evaluation.

Given the lack of labeled datasets in DevOps and the need for a systematic approach to anomaly detection within DevOps workflows, this paper aims to enable time-series anomaly detection within the monitoring systems of these workflows. To this end, we publish a time series dataset named the Westermo Test Systems Performance (WTSP) dataset, collected from Westermo test systems, which includes 24 performance metrics from 19 different test systems, sampled twice a minute over a 30-day period. In order to benchmark the dataset, this paper first makes an effort to identify and label anomalies within the context of the industry since the WTSP dataset does not contain any labels. The labeling was accomplished with the help of feedback from experts in Westermo's DevOps team to ensure that the identified anomalies were accurate and contextually relevant. We then employ six different anomaly detection algorithms on the labeled dataset and conduct a performance evaluation using three different Area Under the Curve (AUC) metrics [12] to establish a baseline for future research on the WTSP dataset. Prior research has demonstrated that the performance of anomaly detection algorithms is influenced by the characteristics of the data and the types of anomalies present [12]. Accordingly, the anomaly detection algorithms in this study were selected considering the specific characteristics of the WTSP dataset and the types of anomalies that can potentially occur in it. The results show that no single algorithm outperformed others across all labeled datasets, emphasizing the importance of selecting an anomaly detection algorithm that best aligns with the dataset's characteristics. Last but not least, this paper introduces and makes available a tool named Performance Data Collection, Monitoring, and Anomaly Detection (PDC-MAD). The tool facilitates the storage, visualization, and monitoring of time series data in DevOps environments as well as the integration of anomaly detection algorithms.

An overview of the DevOps workflow including data collection, data publication, and anomaly detection is depicted in Fig. 1. First, performance data from the servers of many test systems was internally collected and stored in an internal database at Westermo premises. Second, an anonymized, normalized, and open dataset was made available on GitHub using an open license. Third, the PDCMAD tool was developed to conduct anomaly detection using the dataset and to simulate an environment similar to what is used at the company; please refer to Sections III and IV for more details. Finally, this work publishes and benchmarks the WTSP dataset aiming to provide a baseline for future research.

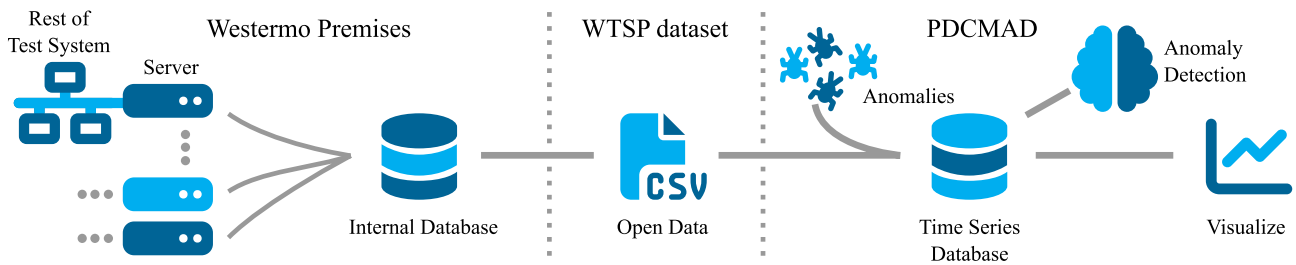


FIGURE 1. An overview of the data collection, publication, and anomaly detection in DevOps workflow.

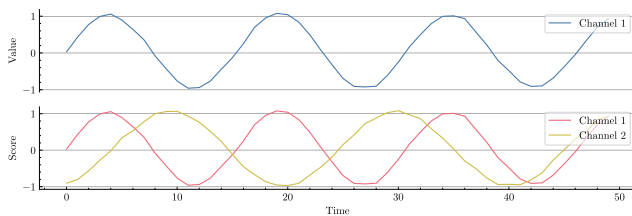


FIGURE 2. Examples of univariate (top) and multivariate (bottom) time series.

The rest of the paper is organized as follows: Section II provides a brief overview of time series anomaly detection and the types of ML algorithms that can be applied for detecting such anomalies, followed by a literature review on anomaly detection and related works within the DevOps context. Next, Section III introduces the Westermo test system and the generated datasets, while Section IV presents the PDCMAD tool. Next, Section V details the labeling approach for the WTSP dataset, the evaluation metrics employed, and the algorithms used for anomaly detection. Section VI then presents the evaluation results of the anomaly detection algorithms on the labeled datasets. Section VII discusses the results obtained, the limitations of the study, and potential directions for future research. Finally, Section VIII concludes the paper.

II. BACKGROUND AND RELATED WORKS

A. TIME SERIES ANOMALY DETECTION

This study focuses on anomaly detection in time series data, which refers to a sequence of data points ordered by time. The data points in the sequence can contain a single variable or multiple variables. A time series comprising a single variable is called *univariate* while a time series consisting of multiple variables is called *multivariate* [5]. Figure 2 shows examples of uni- and multivariate time series. The datasets made available in this paper and those generated upon labeling the datasets are univariate time series.

The task of anomaly detection involves identifying data points that deviate from expected behavior. These anomalies are often classified into different families based on their characteristics [13]. *Point anomalies* refer to the individual data points that do not conform to the general behavior of

the dataset. For example, a single, significantly high CPU usage data point during a period of otherwise low CPU usage in a time series of server metrics. *Collective anomalies* or *sequence anomalies* are a continuous set of anomalous data points. For example, a period of high CPU usage in the same time series of server metrics. *Contextual anomalies* refer to data points that appear anomalous only within a specific context. If a server that normally runs a CPU-intensive task at a specific time of day suddenly shows low CPU usage during that time, that region of the time series could be considered a contextual anomaly.

B. TYPES OF ANOMALY DETECTION ALGORITHMS

Anomaly detection algorithms for time series data are classified into different categories in the literature, depending on the methods employed to detect anomalies. In general, these algorithms can be classified as one of three types depending on the learning method used: unsupervised, supervised, or semi-supervised [14]. *Unsupervised* algorithms do not require labeled data to detect anomalies. Instead, they find the outliers in the data based on some statistical characteristic or distance measure. As such, unsupervised algorithms are well suited when the data is unlabeled or when the anomalies are not known beforehand. The second type, *supervised* algorithms, requires training data labeled as either normal or anomalous [14]. The training data is then typically used to train a model that learns to distinguish between normal and anomalous data points. When presented with new data, the model can then predict whether the data point is normal or anomalous. Given that supervised algorithms require every type of anomaly to be labeled in the training data they are restricted to detecting only the anomalies that are known beforehand. This limits their usefulness in detecting new or unknown types of anomalies, and as such supervised algorithms are rarely used for anomaly detection in practice [12]. In contrast, the third type, *semi-supervised* algorithms, require only the labeling of normal behavior in the time series [14]. Once the normal behavior is learned, the algorithms can detect deviations from it. Unlike supervised algorithms, semi-supervised algorithms can detect unknown and unlabeled anomalies. A disadvantage of semi-supervised algorithms is that any unlabeled anomalies in the training data will be classified as normal behavior. Therefore, the

performance of semi-supervised algorithms depends heavily on the quality of the training data and the contamination level from unlabeled anomalies.

The output of anomaly detection algorithms is typically either binary, classifying each data point as normal or anomalous, or some real value between 0 and 1, which indicates the algorithm's certainty that any given data point is anomalous, where 0 is not certain and 1 is most certain. These techniques are referred to as *label-based* and *score-based* anomaly detection, respectively [2]. In score-based anomaly detection, a threshold t is applied to the score to classify whether a data point is anomalous, allowing the user to adjust the algorithm's sensitivity. All algorithms evaluated in this study are score-based anomaly detection algorithms outputting a real value between 0 and 1.

C. RELATED WORKS

In [11], Wu and Keogh suggest that the current state of anomaly detection benchmarks is flawed. They argue that the standard benchmarks suffer from one of four issues. Firstly, the anomalies in commonly used datasets are too easy to detect. They demonstrate the triviality of detecting anomalies in the Yahoo Benchmark dataset [15] by using a simple statistical method that achieves a competitively high F1 score. Secondly, the density of anomalies in the datasets is too high. Some datasets label almost 50% of the data points as anomalies, which is not necessarily representative of real-world data and has implications for the evaluation of anomaly detection methods. Thirdly, datasets are often mislabeled. The authors demonstrate that the popular Numenta NY Taxi dataset [16], among others, contains regions labeled as non-anomalous which are at least as anomalous as the labeled anomalies. This can lead to overfitting and biased evaluation of anomaly detection methods using these datasets. They suggest that algorithms reported as performing poorly might actually be performing well, but are penalized due to the mislabeled data. Lastly, the location of anomalies in the datasets skew towards the end of the time series. The authors argue that this may be due to the way the datasets are collected as many systems are *run-to-failure* and anomalies are more likely to occur as the system approaches failure. This would reward algorithms that simply predict the last data point as an anomaly. These issues are not unique to the Yahoo Benchmark and Numenta NY Taxi datasets but are present in many commonly used benchmarks. The authors suggest that previous benchmarks utilizing these datasets should be re-evaluated to ensure that the results are valid. This will be taken into consideration when selecting algorithms and datasets for this study. Additionally, the authors provide a list of recommendations for future benchmark datasets. They suggest that the ideal number of anomalies in a dataset is exactly 1. This would ensure that the dataset is representative of real-world data, where anomalies are rare. Using a single anomaly would also make the evaluation of algorithms easier. The evaluation is now binary, either the algorithm detects the anomaly or it does not, which allows for a clear comparison

between algorithms using a simple *accuracy* score. This is not possible with the dataset generated in this paper as it contains multiple anomalies. However, the dataset is representative of real-world data as it is based on data from a real system. The dataset is also labeled by domain experts, which should reduce the risk of mislabeled data.

Schmidl et al. [12] provide a comprehensive overview of the state-of-the-art works in anomaly detection in time series data. As part of their study, they evaluate 71 anomaly detection algorithms on over 900 datasets. The authors collected a wide range of different algorithms and defined different categories depending on the method used by the algorithm. The authors conclude that deep learning algorithms have not yet proven to be competitive with traditional anomaly detection algorithms. Furthermore, they find that no algorithm, or category of algorithms, is consistently the best in all scenarios. This emphasizes the need to test multiple algorithms to find the best one for a specific use case. The authors also find that the performance of most anomaly detection algorithms exhibits high sensitivity to the parameters used to tune the algorithms. They recommend further research into automatic parameter tuning for anomaly detection algorithms. In addition to the results of their evaluation, the authors provide a set of tools for evaluating anomaly detection algorithms. Being open-source, the tools can be utilized for the performance evaluation of anomaly detection algorithms on new datasets. These tools are employed in this paper to assess the performance of the selected algorithms on the WTSP dataset [17].

In addition to the aforementioned works, a plethora of research has been reported in the literature on time series anomaly detection. For instance, Munir et al. [18] propose a Deep Learning (DL)-based anomaly detection approach, DeepAnT, which consists of two key modules: a time series predictor and an anomaly detection module. The predictor, leveraging Convolutional Neural Networks (CNN), forecasts the next value in a time series, which is then analyzed by the anomaly detection module to classify the value as either normal or abnormal. Xu et al. [19] tackle the challenges posed by anomaly contamination in the training data and the lack of prior knowledge about anomalies. They introduce a contamination-tolerant and anomaly-informed unsupervised anomaly detection method. Chauhan and Lee [20] focus on the correlation dependencies in Multivariate Time Series (MTS) data, demonstrating that encoding such correlations significantly improves anomaly prediction accuracy in MTS. For a comprehensive review of DL-based approaches to time series anomaly detection, readers are referred to the survey in [21]. This work highlights recent advancements in DL-based anomaly detection across various application domains and provides practical guidelines for selecting and training DL models to detect anomalies in time series data.

Li et al. [22] released three datasets for AIOps (Artificial Intelligence for IT Operations), focusing on KPI anomaly detection, root cause localization, and failure discovery and diagnosis. Yu et al. [23] introduced Nezha, an interpretable

method for root cause analysis using multi-modal observatory data, with both the data and method made publicly available. The Backblaze hard drive dataset is widely used in the community, representing anomalies through failed drives [24]. Cotroneo et al. [25] published a dataset containing injected faults and error logs from the OpenStack cloud computing platform. Zhu et al. [26] provided the Loghub system log dataset for AI-based log analytics. The WTSP dataset presented in this paper offers a comprehensive dataset from 19 different DevOps test systems with 24 KPIs, contributing to the advancement of anomaly detection research in DevOps workflows.

In order to improve the workflow of DevOps environments and automate the tasks of DevOps teams, recent works are increasingly focusing on incorporating anomaly detection methods. For instance, Hrusto et al. [7] explore the integration of DL solutions into DevOps for anomaly detection in MTS data. The authors incorporate feedback from the development team to label the data, demonstrating that Autoencoders (AEs) perform effectively in anomaly detection tasks within MTS. In [27] as well, the authors utilize DL for anomaly detection and alert reporting in DevOps environments, again emphasizing the role of development feedback for data labeling. Similar to the approaches in [7] and [27], we also incorporate feedback from the DevOps team to label the WTSP dataset. Fawzy et al. [28] extend anomaly detection to two stages in the DevOps life cycle: before production and after staging. They employ local outlier factor and vector autoregression models to identify anomalies at these critical stages. Ali et al. investigate the potential of integrating AI into DevOps workflows to enable predictive analysis and anomaly detection capabilities in [29]. Other approaches have focused on leveraging existing DevOps data for anomaly detection. For instance, Sun et al. [30] use metrics and logs from cloud services to detect anomalies via Support Vector Machines (SVM). Floroiu et al. [31] evaluate a range of ML and statistical models for anomaly detection and root cause analysis in microservice systems, emphasizing the importance of these tasks within DevOps workflows.

Several works present tools or monitoring systems for anomaly detection in DevOps environments. For instance, the TimeEval framework [32] by Wenig et al. facilitates time series data generation, insertion of external time series datasets, and the evaluation of anomaly detection using a wide range of AI algorithms and performance metrics. Moreover, strong Graphical User Interface (GUI) support is provided in the TimeEval framework. In [33], Tarak et al. introduce a tool that applies statistical approaches to detect anomalies in DevOps processes, highlighting its practical applications. Bosch et al. [34] propose a method for generating and managing log data in DevOps environments, which can support anomaly detection efforts. Figalíst et al. [35] developed a feedback system for DevOps environments that combines two ML models to detect anomalies and perform root cause analysis. Thantharate [8] presents the IntelligentMonitor

system, which integrates real-time data collection, intelligent analytics, and automated anomaly detection to streamline DevOps operations. Finally, Islam et al. [36] propose a cloud monitoring platform employing DL models to aid DevOps teams in anomaly detection. While the aforementioned works contribute to anomaly detection in DevOps environments, a major limitation is that these tools are not publicly available to the community, except for [32].

This paper not only publishes a comprehensive dataset collected from a DevOps environment but also benchmarks the dataset by employing various ML and DL models to detect anomalies. Moreover, the dataset labeling process incorporates feedback from industry experts, which is also made publicly available. Additionally, a tool for data storing, visualization, anomaly injection, and anomaly detection is introduced. This holistic approach, encompassing real-world data, an anomaly detection and visualization tool, and the results of anomaly detection algorithms, aims to establish a baseline for future research and foster practical deployments of anomaly detection in DevOps workflows.

III. WESTERMO TEST SYSTEM AND WTSP DATASET

This section first describes the Westermo Test System and then introduces the WTSP datasets collected from these test systems.

At Westermo, the SW, the test framework, and HW are developed in parallel. Prior to release, it must be proven that both the SW and HW work as expected and that the SW can run on the HW. For this purpose, a test framework has been developed and maintained for well over a decade. A central component of the test framework is a fleet of test systems. Many of these test systems have HW in a network topology as illustrated in Figure 3, whereas many virtual test systems with virtual networks are also used (not illustrated). A typical test system has between 6 and 24 devices, equipment for powering the devices on and off, as well as a server driving the nightly testing. New test systems are added many times per year, typically for one of three reasons: to support the testing of new and complex communication protocols (e.g. redundancy protocols that couple rings of devices), to support the testing of new HW products, or to support specific customer cases (e.g. in key customer domains). The test systems are mounted on wagons for easy transport, are given redundant air-conditioning, and the total weight of the test systems is more than a tonne.

During office hours, developers make changes to the code base. This is done in a feature-driven agile process where features are developed in isolation in separate code branches. To support development and debugging, the test systems are used for manual work during the daytime. Before code branches are integrated, they must be tested on the relevant HW and thus on many test systems. This is done with an automated DevOps process where code is automatically compiled, undergoes static analysis, and where the last successful build of the day is used in nightly testing. In parallel with the development of the SW under test, the

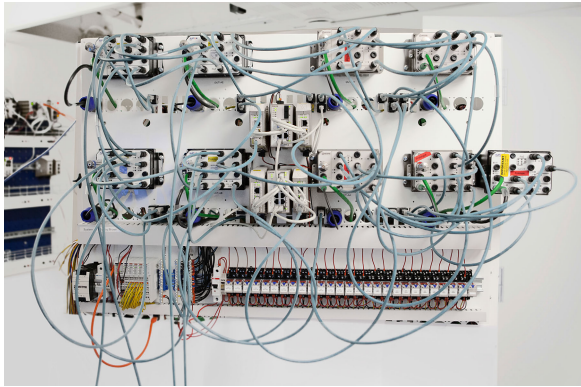


FIGURE 3. Westermo test system.

TABLE 1. Metrics in the dataset, grouped into timestamp, system load, memory, CPU, disk and storage, system, thermal, and server-up metrics.

Metric	Description
timestamp	Seconds into data collection
load-1m	Load over the last 1 minute
load-5m	Load over the last 5 minutes
load-15m	Load over the last 15 minutes
sys-mem-swap-total	Swap size (constant)
sys-mem-swap-free	Available swap
sys-mem-free	Unused memory
sys-mem-cache	Memory used for cache
sys-mem-buffered	Memory used by kernel buffers
sys-mem-available	Memory available to be allocated (free and cache)
sys-mem-total	Total size of memory (constant)
cpu-iowait	Summarized rate of change of seconds spent on waiting for I/O
cpu-system	Summarized rate of change of seconds spent on kernel space threads
cpu-user	Summarized rate of change of seconds spent on user space processes/threads
disk-io-time	Rate of change in time spent on storage I/O operations
disk-bytes-read	Rate of change in bytes read
disk-bytes-written	Rate of change in bytes written
disk-io-read	Rate of change in amount read operations
disk-io-write	Rate of change in amount write operations
sys-fork-rate	Rate of change in the number of forks
sys-interrupt-rate	Rate of change of interrupts
sys-context-switch-rate	Rate of change of context switches
sys-thermal	Average rate of change in measured system temperature (Celsius)
server-up	Values above 0 if the server is up

test system is being maintained and developed in a similar process. During nightly testing, changes by both types of teams are verified and in the mornings, test results can be explored to confirm whether changes were successful or not. This means that, if the test systems are in abnormal states before the test starts, or if they end up in abnormal states during testing, then the test results cannot be trusted,

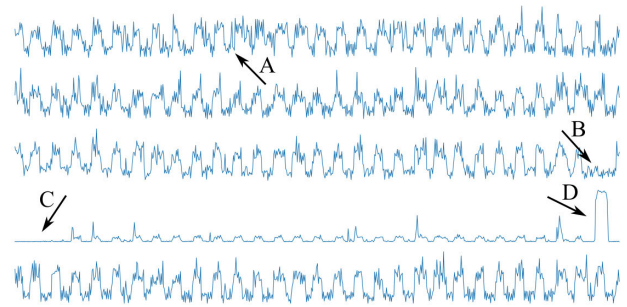


FIGURE 4. The load-15m metric on five systems. Potential anomalies indicated with A (not enough rest?), B (peak missing?), C (missing values?) and D (extreme peak?).

and developers cannot know if their work was successful or not, leading to waste in the form of uncertainty, lost time, effort spent on debugging, etc. This necessitates the integration of anomaly detection approaches within the test systems.

Data collection from the Westermo test systems was carried out within the scope of the ECSEL Joint Undertaking project AIDOaRt.¹ During the project, Westermo started collecting performance metrics from the test systems in a more controlled manner. On the servers of the test systems, a node exporter² was installed, which is a tool for exporting performance data from servers or other PCs. The data was exported from the test systems servers and stored on another server at the Westermo premises in a Prometheus³ monitoring system and time series database. At first, the data was only used for internal purposes. Once it had been internally and anecdotally shown to be valuable, it was decided that a public dataset would be created. After an internal information security risk workshop, a Python script was written for the export. Certain data columns (e.g., specifics on date and exact system) were anonymized, others were copied verbatim (e.g., load and memory metrics), and some were normalized (e.g., CPU and disk metrics). The normalization was motivated by the heterogeneity among the test systems, where some had more CPU cores than others, etc. To simplify reuse, it was decided to export the metrics from each test system into a separate CSV file. In order to facilitate open collaboration within the AIDOaRt project as well as externally with students and the broader research community, Westermo released a partial dataset from 19 test systems on GitHub in late 2023.⁴ Prior to this release, the data was sanitized and documented in [17].

The dataset is stored in 19 CSV files totaling about 360 MB, one file per test system. Each file contains new entries line by line, with various metrics recorded in separate columns. These metrics include system load, memory usage, CPU usage, disk usage, fork rate, temperature, and others, as detailed in Table 1. As an example, the values of one such

¹<https://www.aidoart.eu>

²https://github.com/prometheus/node_exporter

³<https://prometheus.io/>

⁴<https://github.com/westermo/test-system-performance-dataset>

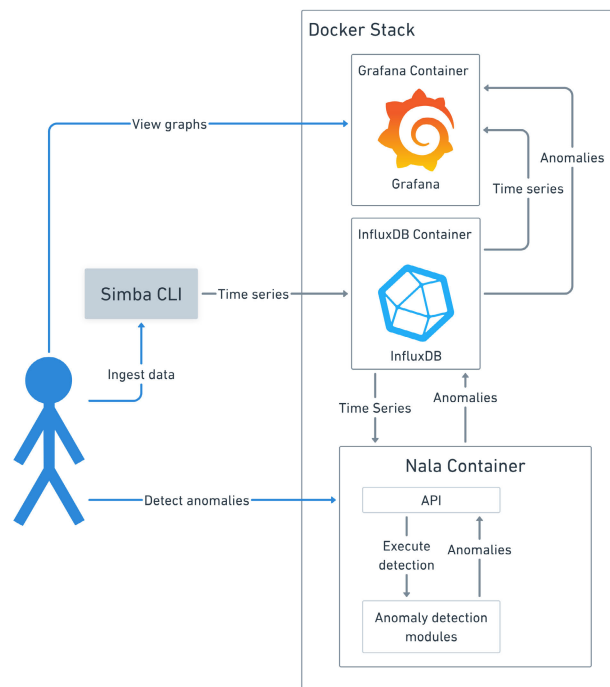


FIGURE 5. A schematic representation of the PDCMAD tool.

metric (load-15m) from five test systems (nr. 10, 12, 14, and 16) are depicted in Figure 4.

IV. PDCMAD TOOL OVERVIEW

Developed as part of a student project, the PDCMAD tool aims to simulate a monitoring system capable of detecting anomalies in a virtual environment closely resembling Westermo's test systems. PDCMAD supports reading data from CSV files, storing it in a database, visualizing the data, injecting anomalies, and detecting anomalies, as illustrated in Figure 1. A schematic representation of the PDCMAD tool, illustrating its overall architecture, is shown in Figure 5. The tool consists of two main components: a backend for ingesting, storing, visualizing, and performing anomaly detection on time series data, and a command-line utility for simulating DevOps test systems by replaying CSV files. To encourage further development and research, the tool has been made publicly available on GitHub.⁵ For detailed instructions on how to use the tool, see the README file in the repository.

As shown in Figure 5, PDCMAD consists of four core components: a time series database (InfluxDB), an anomaly detection API (Nala), a visualization frontend (Grafana), and a simulation tool (Simba). Users, typically developers or testers, interact with the simulation tool to replay CSV files containing time series data. The ingested data is stored in the time series database for subsequent analysis. The anomaly detection API, controlled via HTTP requests, enables the

application of specific detection algorithms on the stored data. Both the raw data and the results of the anomaly detection can be visualized through the frontend.

A. ANOMALY INJECTION

As part of the data ingestion process, users may wish to intentionally inject anomalies into the data to simulate known or potential issues and evaluate the performance of anomaly detection algorithms. To support this, an anomaly injection feature was implemented in PDCMAD. This is achieved by applying a transformation to the data before it is stored in the database. Currently, only basic anomaly injection is supported in PDCMAD, where specific dimensions can be assigned fixed values, such as simulating high CPU usage. As the tool is open-source, further contributions to enhance the anomaly injection capabilities are encouraged.

B. ANOMALY DETECTION

The anomaly detection API is a RESTful API built using Go, designed to enable users to apply anomaly detection algorithms on data stored in the time series database. Controlled via HTTP requests, the API allows users to specify and trigger particular anomaly detection algorithms on selected portions of the dataset. The results of the anomaly detection are subsequently stored in the database and visualized through Grafana. The API is designed for extensibility, making it straightforward to integrate new anomaly detection algorithms by implementing a new endpoint and its corresponding logic within the API.

The future extension of PDCMAD, which will include GUI support and the interpretability of data and ML models, will be linked to the current PDCMAD repository. For more details on the future work of PDCMAD, please refer to Section VII-C.

V. EXPERIMENTAL SETUP

This section outlines the labeling of Westermo test system datasets, the evaluation metrics, and the ML algorithms employed for anomaly detection, which was carried out by the author as part of their thesis in [37].

A. LABELING OF WESTERMO TEST SYSTEM DATASETS

As the datasets collected from Westermo's test systems are unlabeled, a subset of the datasets were labeled to evaluate the performance of the anomaly detection algorithms. To achieve this, two domain experts at Westermo were interviewed and asked to identify anomalies in the datasets. This provided insights into what anomalies might look like in the data and allowed for the creation of a labeled dataset.⁶ Based on the information from the interviews a subset of the Westermo dataset was selected that contained examples of the anomalies that the domain experts classified as important to detect. The anomalies in the subset were then manually labeled to create three labeled datasets for use in the evaluation of

⁵<https://github.com/Kurbitz/PDCMAD>

⁶<https://github.com/Kurbitz/test-system-performance-dataset>

TABLE 2. The testing labeled datasets used in the evaluation. The modification column describes how the datasets were modified to downsample the data.

Measurement	System	# Data points	Real-time	# Anomalies	Modification
CPU-User	14	17262	6 days	313	slice
Sys-Thermal	6	2881	30 days	292	mean (15m)
Sys-Mem-Buffered	1	1454	30 days	31	mean (30m)

TABLE 3. The normal datasets used to train the semi-supervised ML algorithms.

Measurement	System	Length (data points)	Length (real-time)
CPU-User	14	20159	7 days
Sys-Thermal	10	2881	30 days
Sys-Mem-Buffered	17	1441	30 days

the anomaly detection algorithms. The characteristics of the labeled datasets are represented in Table 2. To reduce the complexity of the evaluation only a single type of anomaly was labeled in each dataset. The labeled datasets only contain a portion of the data in the Westermo dataset and only three types of anomalies, serving as a representative sample of the full dataset. In addition to the labeled datasets, a normal dataset was created for each of the three datasets; see Table 3. The normal datasets do not contain any of the anomalies present in the labeled datasets and were used as a training set for semi-supervised ML algorithms.

Anomalies were identified in the measurements *CPU-User*, *Sys-Thermal*, and *Sys-Mem-Buffered* in systems 14, 6, and 1, respectively, within the Westermo test system in Figure 3. The datasets were labeled by extracting the specific measurements that contained the anomalies. A new attribute *is_anomaly* was added to the data to signify whether or not the data point was an anomaly. The anomalies were then labeled as 1 and the rest of the data was labeled as 0. Each dataset is concerned with a single type of anomaly. This way, the datasets can be used to evaluate the performance of anomaly detection algorithms on the specific type of anomaly present in the subset and minimize the risk of undetected anomalies. A description of the features used for anomaly detection together with the rationale behind considering an event as an anomaly is provided below:

- *CPU-User*: The CPU-User metric measures the rate of change of seconds spent on user space processes and threads [17]. The identified anomalies were periods where the CPU usage peaked and stayed high for a long period. Such behavior could indicate that the system is trying to use more CPU resources than what is available. One system was identified as exhibiting this behavior and used to create the first labeled dataset, Table 2.
- *Sys-Thermal*: The Sys-Thermal metric measures the rate of change in temperature (Celsius) of the system [17]. Here, the identified anomaly was a period where the rate of change of temperature was unusually high for a long

period. This was observed in one of the systems and used to create the second labeled dataset, Table 2.

- *Sys-Mem-Buffered*: This metric measures the memory used by kernel buffers [17]. The anomaly identified in this case was a sudden drop in kernel buffer memory usage. This was observed in multiple systems, but only one system was used. This anomaly was not identified as a problem by the experts; however, since it was an unusual pattern in the data it was included in the evaluation to see if the algorithms could detect it. The anomaly was used to create the third labeled dataset, Table 2.

To reduce the size of the datasets, the data was downsampled. The *Sys-Thermal* and *Sys-Mem-Buffered* datasets were downsampled by taking the mean of the data points in 15-minute and 30-minute intervals, respectively. The *CPU-User* dataset was downsampled by extracting a slice of the data with the anomaly present. Downsampling the data was necessary to ensure that the algorithms could be evaluated in a reasonable amount of time. The normal datasets presented in Table 3 were downsampled in the same way as the testing datasets in Table 2.

B. EVALUATION METRICS

Anomaly detection can be framed as a classification problem, where the goal is to classify data points as either *normal* or *anomalous* [14]. The performance of anomaly detection algorithms is usually evaluated similarly to classification algorithms, by using labeled datasets and treating the labeled outliers as the ground truth [38]. The output of the anomaly detection algorithms can then be compared with the ground truth to evaluate their performance. To evaluate score-based anomaly detection algorithms the threshold t that separates anomalies from normal points needs to be considered. If the threshold is placed too high, the algorithm might fail to declare true anomalies as anomalies. Likewise, if the threshold is placed too low, many normal data points might be declared as anomalies. *Precision* and *recall* are therefore

calculated for any given threshold t to compare the relative performance of each threshold. Let, G be the ground-truth anomalies in the labeled dataset D , and $S(t)$ be the set of reported anomalies at t . The *precision* is defined as the *positive predictive value*, i.e., the ratio of reported anomalies that are present in the set of ground-truth anomalies [38].

$$\text{Precision}(t) = \frac{|S(t) \cap G|}{|S(t)|} \quad (1)$$

Similarly, *recall* is defined as the *true positive rate*, the ratio of ground-truth anomalies actually reported as anomalies [38].

$$\text{Recall}(t) = \frac{|S(t) \cap G|}{|G|} \quad (2)$$

The *false positive rate*, also known as the *false alarm ratio*, is the ratio of reported anomalies not labeled as such in the ground-truth [38].

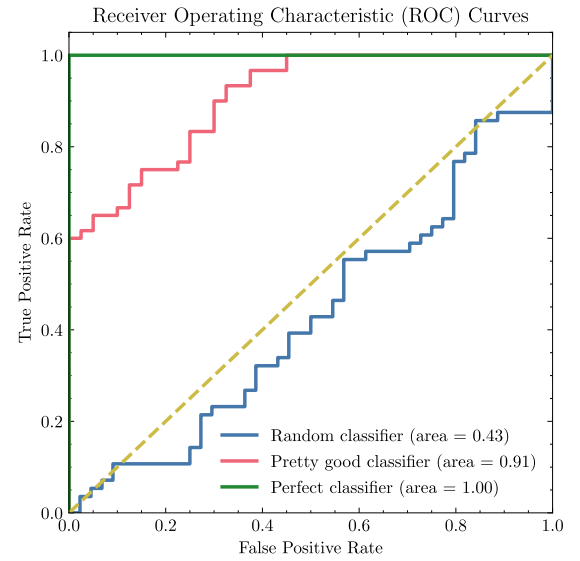
$$\text{False Positive Rate}(t) = \frac{|S(t) - G|}{|D - G|} \quad (3)$$

In order to make the evaluation threshold-independent, *Area Under the Curve* (AUC) metrics are commonly used [12]. These measure the performance of the algorithms as t goes from 0 to 1. In this paper, three metrics are used: the *Area Under the Receiver Operating Characteristic curve* (AUC-ROC) [39], the *Area Under the Precision-Recall Curve* (AUC-PR) [40] and the *Area under the range-based Precision, range-based Recall curve* (AUC- P_{TR_T}) [41]. A perfect classifier would score a 1.0 on each metric.

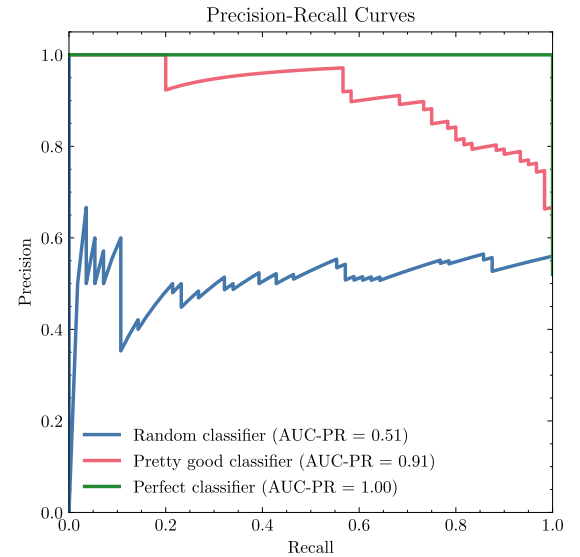
The ROC of a classifier is a two-dimensional graph, plotting the *recall* against the *false positive rate* [38]. The ROC curve is generated by varying the threshold t on the anomaly score and plotting the *true positive rate* against the *false positive rate* for each threshold, as shown in Figure 6a. The curve gives a sense of how well the classifier is able to distinguish between normal and anomalous data points at varying thresholds. For a simpler comparison of ROC curves, the area under the curve is often calculated to reduce it to a single scalar value. The resulting scalar is called the AUC-ROC score. A disadvantage of the AUC-ROC is that it is sensitive to imbalanced classes [40], as is often the case in anomaly detection where the anomalous class is typically rare. Because the ROC curve does not take the false negatives into account, a classifier could achieve a high AUC-ROC by simply classifying all data points as normal.

When evaluating the performance on imbalanced datasets, the AUC-PR metric provides a useful alternative to the AUC-ROC [40] as *recall* takes the false negatives reported by the algorithm into account. The *Precision-Recall curve*, seen in Figure 6b plots a curve between *precision* and *recall* as t goes from 0 to 1. The AUC-PR is the scalar value of the area under this curve.

Both the AUC-ROC and AUC-PR metrics are primarily concerned with individual data points, and as such are



(a) Example of a Receiver Operating Characteristics (ROC) curve.



(b) Example of a Precision-Recall (PR) curve.

FIGURE 6. Example of ROC and PR curves. Note that the axes are different in Figures 6a and 6b.

best suited for datasets containing point anomalies [41]. In the case of sequence anomalies, research has shown that the AUC-PR metric can misrepresent the algorithms' performance [41]. The *Range-based precision* (P_T) and *Range-based recall* (R_T) are new metrics designed specifically for time series anomaly detection where anomalies are often sequences of data points [41]; the AUC- P_{TR_T} is the area under the curve resulting from plotting P_T against R_T .

C. ALGORITHMS FOR EVALUATING ANOMALY DETECTION

Table 4 lists the algorithms evaluated in this study. The algorithms were chosen based on their performance in previous studies. Efforts were made to include algorithms

TABLE 4. The algorithms evaluated in anomaly detection.

Algorithm	Method Family	Type
DWT-MLEAD [42]	Signal Analysis	Unsupervised
Grammar Viz [43]	Data Mining	Unsupervised
k-Means [44]	Classic ML	Unsupervised
LSTM-AD [45]	Deep Learning	Semi-supervised
Subsequence IF [46] [32]	Outlier Detection	Unsupervised
Subsequence LOF [47] [32]	Outlier Detection	Unsupervised
Reference Algorithm	NA	NA

from the different method families defined by Schmidl et al. [12]. *Outlier Detection* is represented twice due to both performing well in the literature. All algorithms were available as open-source implementations as part of the TimeEval framework [32]. A brief summary of each algorithm is provided below, while readers are referred to the corresponding references for further details, as an in-depth discussion is beyond the scope of this paper.

- *k-Means*: An anomaly detection algorithm using the k-means clustering algorithm. Similar patterns in the data are grouped by their distance from the cluster center. Anomalies are the patterns that are farthest from the cluster center [44].
- *Subsequence LOF*: An adaptation of the Local Outlier Factor (LOF) algorithm, made to work specifically with subsequences of time series data. The algorithm calculates a score for each subsequence that represents the deviation of density within the subsequence compared to the density of its neighbors [47].
- *Subsequence IF*: An adaptation of the Isolation Forest algorithm, modeled to work specifically with subsequences of time series data. Isolation Forest works by recursively partitioning the data into smaller subsets until the data points most different from the rest are isolated [46].
- *DWT-MLEAD*: An anomaly detection algorithm that uses the Discrete Wavelet Transform (DWT) to represent the time series as a set of waves (wavelets). The algorithm then uses a sliding window that moves over the time series and separates unusual patterns from the rest of the data [42].
- *Grammar Viz*: An anomaly detection algorithm that uses the Symbolic Aggregate Approximation (SAX) representation of the time series data. It discretizes the data into symbols and uses the Sequitur algorithm to infer a grammar to represent the patterns in the data. Unique patterns are considered anomalies [43].
- *LSTM-AD*: An anomaly detection algorithm that uses a Long Short-Term Memory (LSTM) neural network to learn the normal behavior of the time series. The model, trained on the normal data, then predicts the next data points in the time series. The error between the predicted data points and the actual data points is used to detect anomalies [45].

D. REFERENCE ALGORITHM (RA)

We adopted a simple threshold-based method as a baseline that detects anomalies by comparing the data points to a threshold. The threshold was set to the value equal to the 95th percentile of the data in each dataset. Any data point above the threshold was classified as an anomaly. As the anomaly in the *Sys-Mem-Buffered* dataset was an unusually low data point, as opposed to the other datasets where the anomalies were unusually high data points, the threshold was set to the 5th percentile in that dataset (or the 95th percentile of the inverted data). Implemented in Python with the NumPy library [48], the pseudo-code of the reference algorithm is provided in Algorithm 1.

Algorithm 1 Reference Algorithm (RA)

Input: X - The data to classify
Output: y - The classification of the data
procedure ThresholdReferenceAlgorithm(X)
 $threshold \leftarrow \text{percentile}(X, 95)$
 $y \leftarrow \text{map}(\lambda x : 1 \text{ if } x > threshold \text{ else } 0, X)$
 return y
end procedure

E. HYPERPARAMETER TUNING

An informal grid search was conducted to determine the best hyperparameters for the algorithms. Each parameter was given a range of values and an exhaustive search was performed over the different combinations of parameters. As the number of tests needed to exhaustively search the parameter space grows exponentially with the number of parameters, the search space was limited to 10 different values for each parameter. The values were chosen based on the literature and the default values of the algorithms. The parameters that resulted in the highest AUC-PrR_T score were selected for the final evaluation. The tuning was performed per dataset so that the best parameters for each dataset could be found. The final parameters for each algorithm and dataset are shown in Table 10, Table 11, and Table 12 in Appendix A. Any parameter not listed in the table was set to the default value of the algorithm. Please note that the parameters were tuned using the test sets as no validation set was available.

VI. EVALUATION OF ANOMALY DETECTION RESULTS

The evaluation was performed using Python 3 and the TimeEval framework [32], an open-source framework for evaluating time series anomaly detection algorithms. The framework provides built-in support for running anomaly detection algorithms and evaluating their performance on labeled datasets using various metrics. Moreover, TimeEval uses pre-configured Docker containers with implementations of the algorithms to ensure reproducibility. Three separate evaluations were performed, one for each dataset. Algorithms were evaluated using the AUC-ROC, AUC-PR, and AUC-PrR_T metrics, described in Section V-B. The evaluation was carried out on a workstation with an AMD Ryzen 5

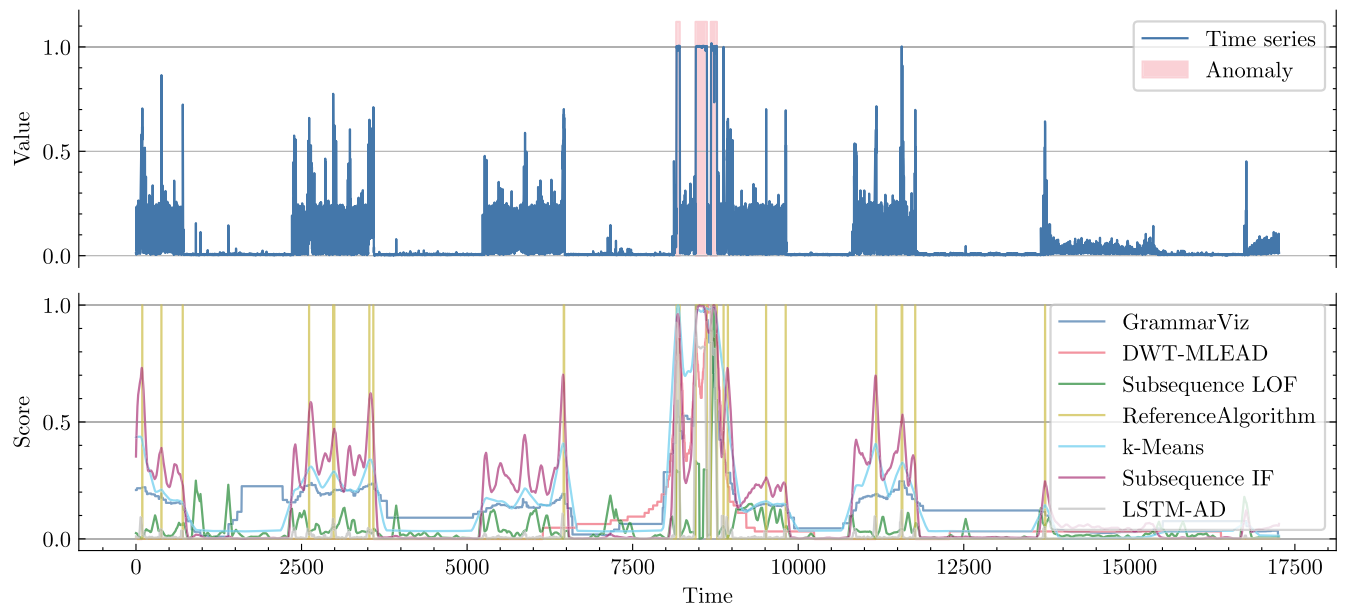


FIGURE 7. Upper plot: The *CPU-User* dataset with the ground-truth anomalies marked in red. Lower plot: The anomaly scores of the algorithms. A zoomed-in view of the anomalous region is depicted in Figure 10, Appendix B for a clearer visualization.

3700X and 32 GB of RAM. No parallelization or distributed computing was used, nor was there any GPU acceleration.

A. RESULTS

The results on each dataset are presented separately, followed by a summary of the results.

1) CPU-USER

The results for the *CPU-User* dataset are shown in Table 5, sorted by AUC- P_{TRT} score from lowest to highest. Although most algorithms achieved a high AUC-ROC score, the AUC-PR and AUC- P_{TRT} scores varied significantly. Only two algorithms, k-means and Subsequence IF, achieved an AUC- P_{TRT} score above 0.9. This indicates a high number of false positives in the other algorithms. As shown in Figure 7, most algorithms struggle with the seasonality in the dataset and incorrectly report short spikes of high CPU usage as anomalies. This is expected, as anomaly detection algorithms are typically good at detecting extreme point anomalies, but struggle with more complex anomalies [12].

The reference algorithm achieves high AUC-ROC and AUC-PR scores but has the lowest AUC- P_{TRT} score of all algorithms. This indicates that the reference algorithm has a high number of false positive regions, as shown in Figure 7. Its high AUC-PR score can be attributed to the length of the false positive regions. Since the ground-truth anomalies are much longer than the short spikes of high CPU usage, the reference algorithm is able to capture a large portion of the ground-truth anomalies. Since the reference algorithm is a simple threshold-based method, it is not able to distinguish between the short spikes of high CPU usage and the ground-truth anomalies. This is reflected in the low AUC- P_{TRT} score.

TABLE 5. Results of the evaluation on the *CPU-User* dataset showing the AUC- P_{TRT} , AUC-PR, and AUC-ROC scores for each algorithm, sorted by AUC- P_{TRT} score.

Algorithm	AUC- P_{TRT}	AUC-PR	AUC-ROC
Reference Algorithm (RA)	0.553	0.960	0.999
DWT-MLEAD	0.642	0.559	0.994
Subsequence LOF	0.664	0.806	0.852
LSTM-AD	0.746	0.862	0.998
Grammar Viz	0.835	0.873	0.997
Subsequence IF	0.934	0.996	1.0
k-means	0.944	0.878	0.998
Average (RA excluded)	0.794	0.829	0.973

The performance of LSTM-AD on this dataset is interesting. The algorithm achieves a high AUC-ROC score, a lower AUC-PR score, and a mediocre AUC- P_{TRT} score. When visually inspecting the anomaly scores in Figure 11 (Appendix B) it seems to have almost perfectly captured the ground-truth anomalies. On closer inspection, it seems as though the scores are slightly shifted in time, starting and ending slightly earlier than the ground-truth anomalies. Despite this, the algorithm has among the highest confidence in its predictions, staying close to 0 for normal data points and close to 1 for anomalies. This is likely due to LSTM-AD being semi-supervised, it has learned through training to ignore the seasonal spikes in the data.

2) SYS-THERMAL

The results of the evaluation on the Sys-Thermal dataset are shown in Table 6, sorted by AUC- P_{TRT} score from lowest to highest. Every algorithm, except the reference

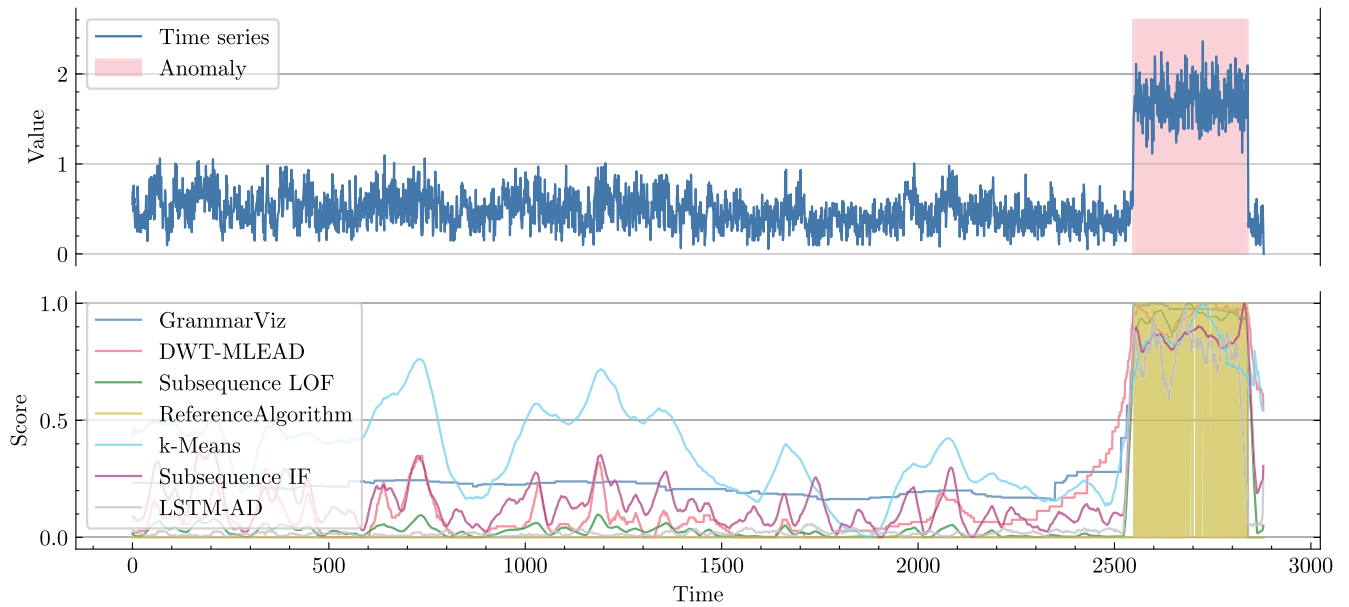


FIGURE 8. Upper plot: The *Sys-Thermal* dataset with the ground-truth anomalies marked in red. Lower plot: The anomaly scores of the algorithms. A zoomed-in view of the anomalous region is depicted in Figure 12, Appendix B for a clearer visualization.

TABLE 6. Results of the evaluation on the *Sys-Thermal* dataset showing the AUC- $P_T R_T$, AUC-PR, and AUC-ROC scores for each algorithm, sorted by AUC- $P_T R_T$ score.

Algorithm	AUC- $P_T R_T$	AUC-PR	AUC-ROC
Reference Algorithm (RA)	0.780	0.780	0.755
k-means	0.848	0.963	0.995
LSTM-AD	0.960	0.988	0.998
Subsequence IF	0.974	0.997	0.999
DWT-MLEAD	0.988	0.990	0.988
Subsequence LOF	0.996	1.0	0.999
Grammar Viz	1.0	1.0	1.0
Average (RA Excluded)	0.961	0.990	0.997

algorithm, achieved relatively high scores in all metrics. The difference in performance between the best-performing algorithms is less pronounced than on the CPU-User dataset and is small enough to be negligible. This is likely due to the simplicity of the anomalies in the dataset. As shown in Figures 8, the anomalous region is clearly defined and easily distinguishable from the normal data points.

The reference algorithm did not perform as well as expected. This is likely due to the threshold being set too low, resulting in a high number of false negatives. With a more appropriate threshold, the reference algorithm could likely achieve a higher, almost perfect, score. This shows the difficulty in choosing the right value for simple threshold-based algorithms.

The best-performing algorithm on this dataset is Grammar Viz. The algorithm achieved a perfect score in all metrics, indicating that it, at some threshold, perfectly separates the normal data points from the anomalies.

Although k-Means was the best-performing algorithm on the CPU-User dataset, it does not perform well on the *Sys-Thermal* dataset, reference algorithm excluded. The algorithm incorrectly reports two other regions as anomalous, likely due to some other pattern in the data that the algorithm has picked up on. This is an interesting result and shows how the performance of anomaly detection algorithms can vary significantly depending on the dataset and the anomalies one is trying to detect.

Again, LSTM-AD correctly identifies the ground-truth anomalies, but the scores are slightly shifted in time; it starts and ends slightly earlier than the ground-truth anomalies (see Figure 13 in Appendix B). In this case, the performance is not impacted as heavily as on the CPU-User dataset, likely due to the *Sys-Thermal* dataset being smaller, meaning fewer data points are shifted, affecting the AUC- $P_T R_T$ scoring less. The performance of LSTM-AD on this dataset is similar to the performance on the CPU-User dataset, indicating that the algorithm is robust to different types of anomalies.

3) SYS-MEM-BUFFERED

The results of the evaluation on the *Sys-Mem-Buffered* dataset are shown in Table 7, sorted by AUC- $P_T R_T$ score from lowest to highest. The results show that, again, most algorithms perform well on the dataset, with all algorithms achieving almost perfect AUC-ROC scores, high AUC- $P_T R_T$ scores, and even higher AUC-PR scores. As shown in Figure 9, every algorithm correctly identifies the ground-truth anomalies as the most anomalous regions in the dataset. As with the *Sys-Thermal* dataset, the anomalies in the *Sys-Mem-Buffered* dataset are clearly defined and easily distinguishable from the normal data points by visual

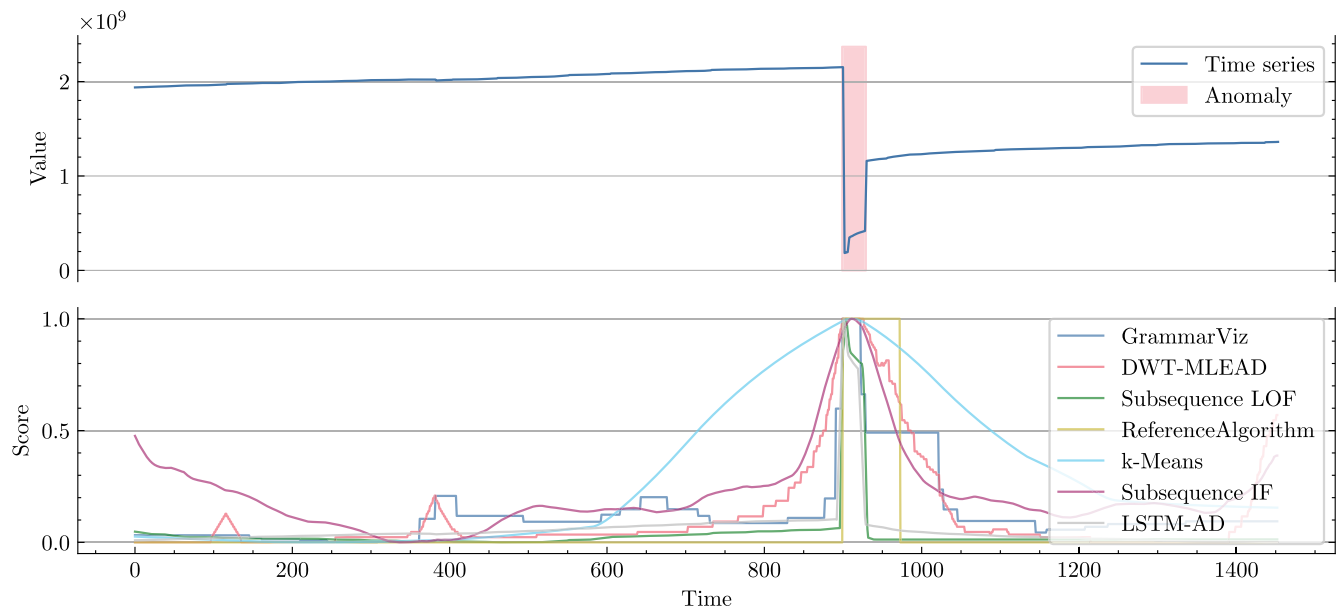


FIGURE 9. Upper plot: The Sys-Mem-Buffered dataset with the ground-truth anomalies marked in red. Lower plot: The anomaly scores of the algorithms. A zoomed-in view of the anomalous region is depicted in Figure 14, Appendix B for a clearer visualization.

TABLE 7. Results of the evaluation on the *Sys-Mem-Buffered* dataset showing the AUC- $P_T R_T$, AUC-PR, and AUC-ROC scores for each algorithm, sorted by AUC- $P_T R_T$ score.

Algorithm	AUC- $P_T R_T$	AUC-PR	AUC-ROC
Reference Algorithm (RA)	0.690	0.690	0.969
LSTM-AD	0.861	0.937	0.994
k-means	0.941	0.985	1.0
Subsequence LOF	0.978	0.999	1.0
Grammar Viz	0.984	0.984	0.999
Subsequence IF	0.992	0.999	1.0
DWT-MLEAD	0.999	0.999	1.0
Average (RA)	0.959	0.984	0.999

inspection. This is reflected in the high scores of the algorithms.

The reference algorithm achieves a high AUC-ROC score, but a low AUC- $P_T R_T$ score. This indicates that the reference algorithm correctly identifies the ground-truth anomalies, but also reports a number of normal data points as anomalies. This is, again, due to the threshold not being set correctly to exactly separate the normal data points from the anomalies. There is a threshold, at which the reference algorithm would achieve a perfect score, but such a threshold would have to be set manually for each system, limiting threshold-based algorithms’ usefulness in practice.

The LSTM-AD scores are slightly shifted in this case as well, as seen in Figure 15, Appendix B. This is reflected in the AUC-ROC and AUC-PR scores being higher than the AUC-P_TR_T score. Despite this, the algorithm performs well on the dataset, correctly identifying the ground-truth anomalies. The performance of LSTM-AD on this dataset is similar to the performance on the other datasets, indicating

again that the algorithm is robust to different types of anomalies.

As most algorithms achieve high scores across all metrics on the Sys-Mem-Buffered dataset, the differences in performance among the top algorithms are negligible. This is likely due to the simple nature of the anomalies in the dataset and the clear distinction between normal and anomalous data points.

B. SUMMARY

The results, averaged over all datasets, are shown in Table 8 along with the average of all algorithms excluding the reference algorithm. The results show that most algorithms perform well on the datasets, averaging above 0.9 in all metrics. Especially, classical models such as Subsequence IF, k-means, and Grammar Viz outperform the deep learning model LSTM-AD by a significant margin in terms of AUC- P_{TR} , with scores of 0.96, 0.94, and 0.91, respectively. This performance is likely due to the relatively small labeled dataset, where the lightweight nature of classical methods allows them to perform effectively. However, this should not be viewed as a general conclusion about classical versus deep learning models in the DevOps environment. Instead, the choice of ML/DL models should be based on the dataset characteristics and resource constraints.

The average execution and training times for the algorithms are shown in Table 9, providing insights into the feasibility of real-time anomaly detection. The reference algorithm is the fastest by several orders of magnitude, which is expected due to its simplicity. The execution time of the other algorithms ranges from 2 to 8 seconds in execution time, with Subsequence LOF being the slowest. As LSTM-AD is the only algorithm that requires training and therefore, has the

TABLE 8. Results of the evaluation on all datasets showing the average AUC- $P_T R_T$, AUC-PR, and AUC-ROC scores for each algorithm, sorted by AUC- $P_T R_T$ score.

Algorithm	AUC- $P_T R_T$	AUC-PR	AUC-ROC
Reference Algorithm (RA)	0.674	0.810	0.908
LSTM-AD	0.856	0.929	0.997
DWT-MLEAD	0.876	0.849	0.998
Subsequence LOF	0.879	0.935	0.951
k-means	0.911	0.942	0.998
Grammar Viz	0.940	0.953	0.999
Subsequence IF	0.967	0.997	1.000
Average (RA excluded)	0.905	0.934	0.990

TABLE 9. Average execution and training times.

Algorithm	Execution time (s)	Training time(s)
Reference Algorithm	0.0001	NA
LSTM-AD	2.0	39.9
DWT-MLEAD	2.6	NA
Grammar Viz	3.3	NA
Subsequence IF	3.4	NA
k-means	3.6	NA
Subsequence LOF	7.7	NA

longest total time, 41 seconds. Training time is not applicable for the other algorithms as they are unsupervised.

For real-time anomaly detection, algorithms with lower execution time, such as LSTM-AD, DWT-MLEAD, or Subsequence-IF, could be more suitable depending on the latency constraints of the deploying environment. Moreover, real-time processing depends on many other factors, such as data stream rate, computational resources, parallelization, and GPU acceleration. As no such measures were taken in this work, future work could include optimizing the computational efficiency of these algorithms and evaluating their performance under real-time operational constraints.

VII. DISCUSSION, LIMITATIONS, AND FUTURE WORK

A. DISCUSSION

The paper presents the WTSP dataset and benchmarks it by carrying out a performance evaluation of anomaly detection in DevOps workflows using six different algorithms. To this end, a subset of the unlabeled WTSP dataset was labeled to identify anomalies in the *CPU-User*, *Sys-Thermal*, and *Sys-Mem-Buffer* metrics. The *CPU-User* dataset stands out as the most challenging dataset, with the algorithms performing worse on this dataset compared to the other two on average. This suggests a higher degree of complexity among the anomalies in the *CPU-User* dataset. The performance on the other two datasets is more consistent, with the algorithms performing similarly on the *Sys-Thermal* and *Sys-Mem-Buffer* datasets. Although some algorithms performed well on all datasets, notably Subsequence IF and GrammarViz, no single algorithm consistently performed the best on all

datasets, which is in line with the conclusions drawn in [12]. This emphasizes the need to test multiple algorithms to find the one(s) that fit the specific use case.

Although the LSTM-AD reported lower AUC- $P_T R_T$ scores than the other algorithms, it consistently performed well on the datasets in other metrics. The discrepancy in the AUC- $P_T R_T$ score is likely due to the slight shift in time of the anomaly scores. Regardless, this does not limit the usefulness of the algorithms much in practice, as the general area of the anomalies is still correctly identified. The consistently high scores across different datasets and anomalies demonstrate the robustness of semi-supervised DL algorithms in anomaly detection. Being able to train the algorithm on normal data points allows practitioners to use the algorithm without identifying specific anomalies beforehand. This opens up the possibility of using anomaly detection algorithms on systems where the normal behavior is known, but potential anomalies are not.

Without loss of generality, while deep learning models such as LSTM-AD excel in accuracy, they come with reduced interpretability. In contrast, classical ML models like Subsequence LOF, Grammar Viz, and k-means offer high interpretability, albeit with potentially lower accuracy. Additionally, when selecting AI algorithms for DevOps contexts, the high computational resources required by deep learning models must be considered, as classical ML models tend to be more lightweight. Furthermore, deep learning models typically require large labeled datasets to generalize effectively, which can be challenging given the scarcity of labeled datasets in DevOps environments [10].

B. LIMITATIONS

Certain limitations apply to the generalizability of the results presented in this study. For instance, the amount of datasets produced and anomalies tested is too low to confidently say that the results are generalizable across the entire Westernmo dataset or the DevOps domain in general. Further effort is required to create more labeled datasets containing different types of anomalies, sourced from multiple systems and contexts, to ensure the quality of the results. This would also alleviate the second limitation, i.e., the lack of validation sets to tune the algorithms. As the parameters were tuned using the test sets, it is possible that the algorithms are overfitting to the test sets and would not perform as well on unseen, similar data. Moreover, this limitation on the datasets can potentially impact the bias and variance trade-offs. Particularly, as the deep learning models, such as LSTM-AD, are known to exhibit high variance as the model complexity increases [49], it may lead to overfitting on the test data. On the other hand, the classical models, which tend to have higher bias, may generalize well on the smaller datasets such as those used in this study; this is likely the reason for the high performance of the classical models compared to the deep learning models in this study; see Table 8.

Furthermore, as the anomalies were identified through visual inspection, it is possible that the anomalies are trivial

TABLE 10. The hyperparameters used for each algorithm on the *CPU-User* dataset.

Algorithm	Parameters
k-means	n_clusters=2, anomaly_window_size=200
DWT-MLEAD	quantile_epsilon=0.02, start_level=3
Grammar Viz	alphabet_size=4, anomaly_window_size=200, paa_transform_size=3
Subsequence LOF	n_neighbors=50, window_size=60
Subsequence IF	window_size=75 n_trees=100
LSTM-AD	window_size=27

to detect. This could explain the high performance of the algorithms in the evaluation. Using out-of-band (OOB) information (logs, process data, etc.), as suggested by Wu and Keogh [11], could help produce high-quality datasets with a variety of non-trivial anomalies.

C. FUTURE WORK

In the interviews conducted to yield the labeled datasets in this paper, both domain experts stated that correlations between different systems and metrics are important to monitor; therefore, multivariate anomaly detection could be an interesting approach to explore in future work. This would require further effort in the labeling process, as the anomalies would be more complex to detect. Again, using OOB information could be beneficial to ensure that the anomalies are not trivial. The reason why this was not explored in this study is both due to the lack of availability of open OOB data and the scope of the study. Finding, labeling, and preprocessing suitable measurements to perform such an evaluation could be an interesting area for further investigation. It has the potential to reveal interesting contextual anomalies in the WTSP dataset, such as deviations from the collective seasonal pattern.

In addition to multivariate anomaly detection, future work could investigate the ensemble methods [50], which this study lacks; analyzing the impact of combining statistics-based and learning-based methods on anomaly detection requires further investigation. Another interesting research direction could be to examine the use of other data sets, perhaps from other aspects of DevOps and variables not explored in the WTSP dataset, e.g. duration(s) for compilation, automated parsing of log files [29], [34], the actual verdicts of the testing, etc. Furthermore, conducting statistical analysis, such as the Wilcoxon rank-sum test, would provide stronger evidence on whether one model significantly outperforms another. Additionally, incorporating confidence intervals into model performance scores would help quantify variability and enhance the robustness of the results.

The current version of the PDCMAD tool includes only the Isolation Forest algorithm for anomaly detection, with the primary focus on facilitating the integration of new algorithms. Future updates will enhance usability by enabling various types of anomaly injection and simplifying the integration of external ML libraries, with all PDCMAD components transitioning to Python. Moreover, considering practical

deployment aspects, support for both batch processing and real-time data streams will be provided, and GUI support will be introduced to improve user experience. Most importantly, we plan to facilitate data and model interpretability in the next version by incorporating feature importance analysis and Shapley Additive Explanations (SHAP) to enable deeper insights into anomaly detection decisions made by the interpretable ML models. To streamline the installation and setup process, future releases will continue to be provided as a Docker container, as is the case with the current version. All updates and enhancements will be documented in the PDCMAD repository's release notes, and if a separate repository is created, a reference link will be included in the README file.

While the current version of PDCMAD has been tested on the WTSP datasets using the IF algorithm, its ability to handle large-scale industrial logs or high-frequency real-time data is yet to be investigated. Moreover, the computational constraints for such scalability, as well as the possibility of parallel processing and incorporating GPU acceleration or distributed computing frameworks, require further research to deploy anomaly detection in industrial settings.

Finally, the WTSP dataset is a comprehensive collection of time series data from DevOps test systems, which has the potential to advance anomaly detection research in DevOps workflows. Given the growing interest in this field in industries utilizing DevOps, the dataset can be leveraged to explore novel approaches to anomaly detection, enabling higher efficiency and accuracy in DevOps systems.

VIII. CONCLUSION

This paper presents a ground truth time series dataset from DevOps test systems and benchmarks it to establish a baseline for future research on integrating time series anomaly detection in DevOps environments. In addition, a tool named PDCMAD is made available to the community to facilitate data storage, visualization, anomaly injection, and detection. Further, in this paper, we seek to identify common problems in DevOps test systems that can be detected as anomalies in the time series performance metrics produced by the systems using anomaly detection algorithms. In identifying the anomalies and labeling the time series data, interviews were conducted with domain experts, yielding three new, labeled datasets. Further, an evaluation of six different anomaly detection algorithms, chosen based on

TABLE 11. The hyperparameters used for each algorithm on the *Sys-Thermal* dataset.

Algorithm	Parameters
k-means	n_clusters=120, anomaly_window_size=70
DWT-MLEAD	quantile_epsilon=0.16, start_level=1
Grammar Viz	alphabet_size=2, anomaly_window_size=200, paa_transform_size=3
Subsequence LOF	n_neighbors=800, window_size=25
Subsequence IF	window_size=25, n_trees=150
LSTM-AD	window_size=25

TABLE 12. The hyperparameters used for each algorithm on the *Sys-Mem-Buffered* dataset.

Algorithm	Parameters
k-means	n_clusters=2, anomaly_window_size=320
DWT-MLEAD	quantile_epsilon=0.08, start_level=1
Grammar Viz	alphabet_size=4, anomaly_window_size=100, paa_transform_size=3
Subsequence LOF	n_neighbors=512, window_size=4
Subsequence IF	window_size=64, n_trees=32
LSTM-AD	window_size=8

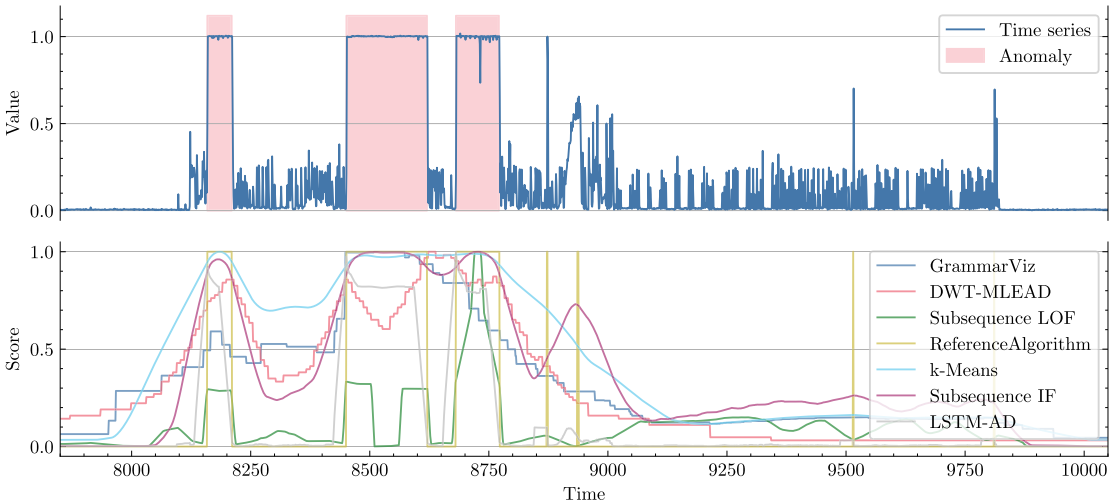


FIGURE 10. Zoomed-in view of the *CPU-User* dataset (corresponding to Figure 7) and the algorithms’ anomaly scores, providing a closer visualization of the anomalous regions and the scores.

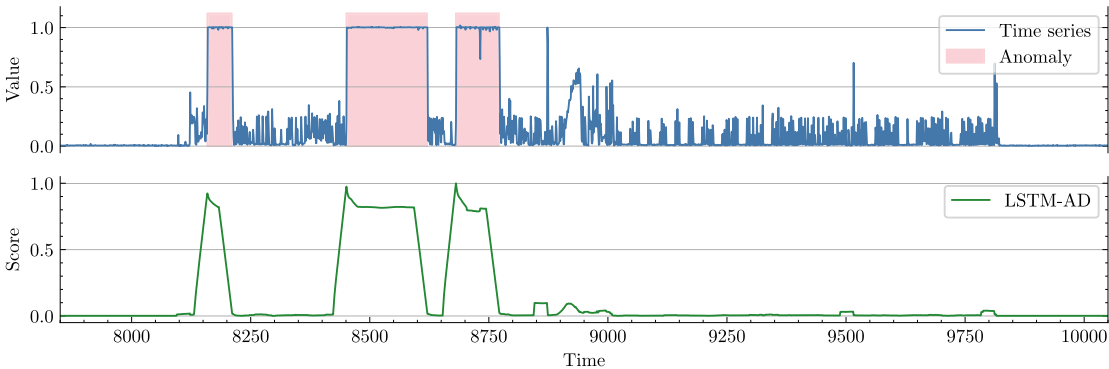


FIGURE 11. Only the LSTM-AD anomaly scores from Figures 7 and 10 plotted against the *CPU-User* dataset. The red regions indicate the ground-truth anomalies. Zoomed in to show the anomalous regions.

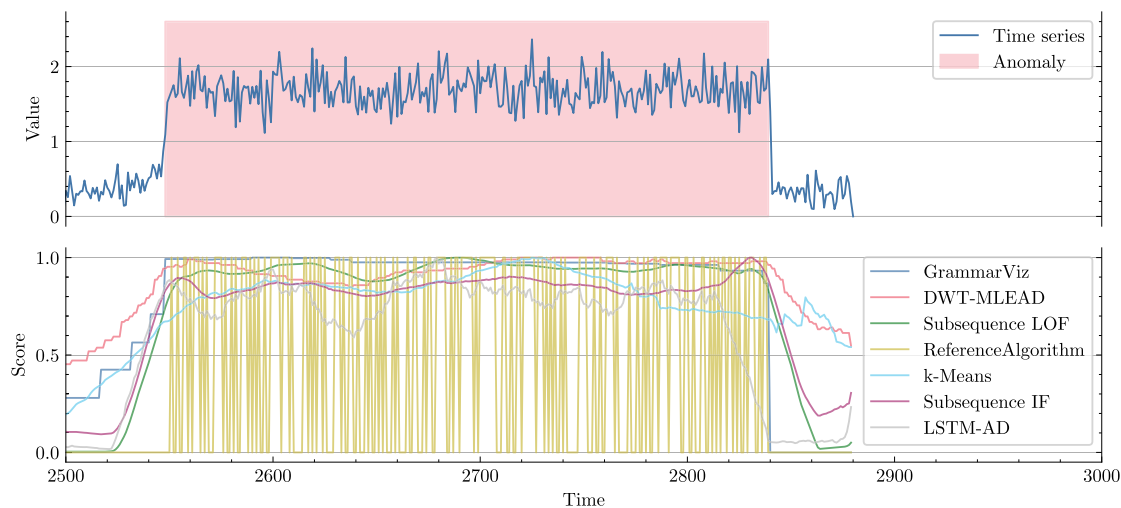


FIGURE 12. Zoomed-in view of the *Sys-Thermal* dataset (corresponding to Figure 8) and the algorithms' anomaly scores, providing a closer visualization of the anomalous regions and the scores.

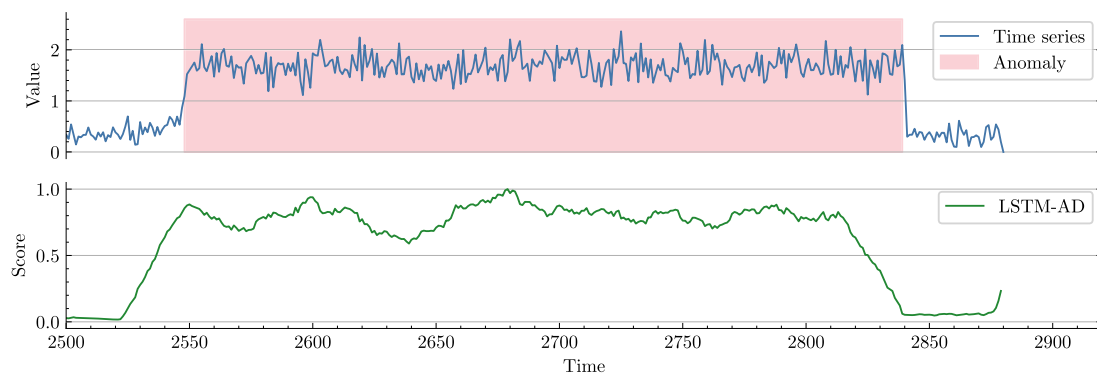


FIGURE 13. Only the LSTM-AD anomaly scores from Figures 8 and 12 plotted against the *Sys-Thermal* dataset. The red regions indicate the ground-truth anomalies. Zoomed in to show the anomalous regions.

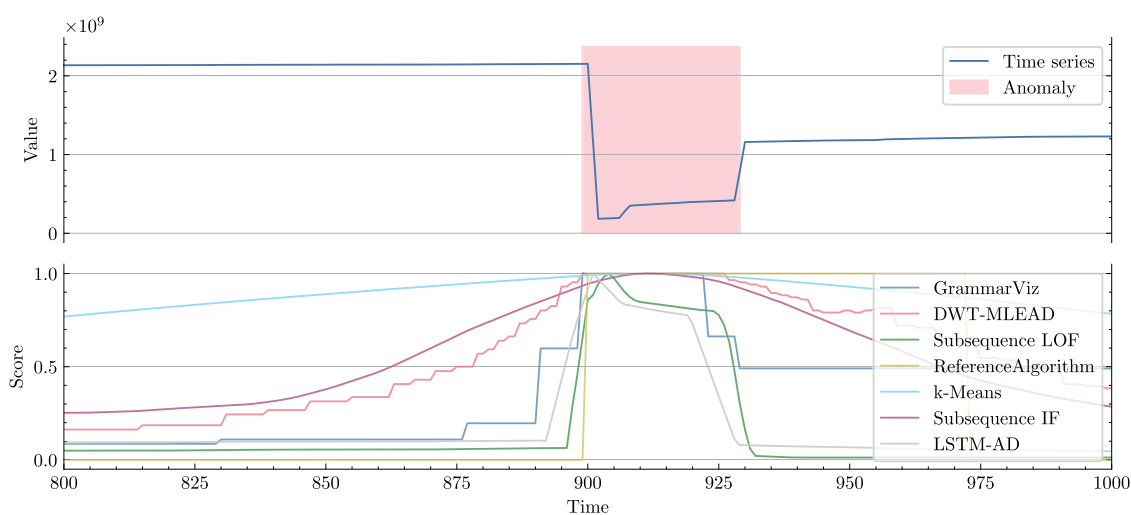


FIGURE 14. Zoomed-in view of the *Sys-Mem-Buffered* dataset (corresponding to Figure 9) and the algorithms' anomaly scores, providing a closer visualization of the anomalous regions and the scores.

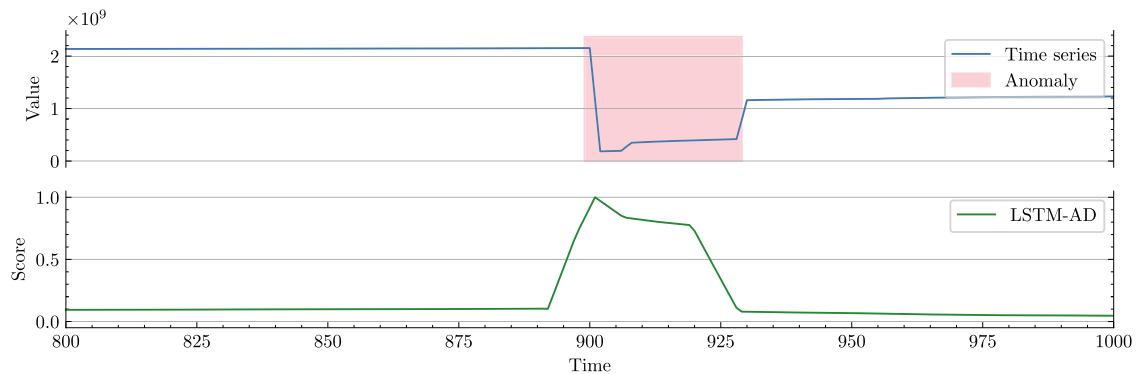


FIGURE 15. Only the LSTM-AD anomaly scores from Figures 9 and 14 plotted against the *Sys-Mem-Buffered* dataset. The red regions indicate the ground-truth anomalies. Zoomed in to show the anomalous regions.

their performance in previous studies, was then conducted. A simple threshold-based algorithm, similar to those used in production, was included as a benchmark. The evaluation results showed that the algorithms were able to detect the anomalies identified by the domain experts, but that no single algorithm consistently performed the best on all datasets. On the other hand, the reference algorithm struggled with false positives. The results suggest that anomaly detection algorithms can be useful in a DevOps context, but that no silver bullet exists. The choice of algorithm and the parameters used must be motivated by the context in which the algorithms are used. The results are in line with previous research which found that no algorithm is consistently the best in all scenarios.

APPENDIX A ALGORITHM HYPERPARAMETERS

See Tables 10–12.

APPENDIX B EVALUATION PLOTS

See Figures 10–15.

ACKNOWLEDGMENT

The authors would like to thank the developers of PDCMAD (in alphabetical order): Oscar Einarsson, Gustav Kånåhols, Tahira Nishat, Fredrik Nygård, Fernando Revillas, and Adam Segerström.

REFERENCES

- [1] G. Gruver, *Start and Scaling Devops in the Enterprise*. Pennsauken, NY, USA: Bookbaby, Dec. 2016.
- [2] V. Chandola, A. Banerjee, and V. Kumar, “Anomaly detection,” *ACM Comput. Surv.*, vol. 41, no. 3, pp. 1–58, Jul. 2009.
- [3] R. Foorthuis, “On the nature and types of anomalies: A review of deviations in data,” *Int. J. Data Sci. Anal.*, vol. 12, no. 4, pp. 297–331, Oct. 2021.
- [4] P. J. Brockwell and R. A. Davis, *Time Series: Theory and Methods*. Cham, Switzerland: Springer, 1991.
- [5] A. Blázquez-García, A. Conde, U. Mori, and J. A. Lozano, “A review on outlier/anomaly detection in time series data,” *ACM Comput. Surveys*, vol. 54, no. 3, pp. 1–33, Apr. 2022.
- [6] W. Pourmajidi, J. Steinbacher, T. Erwin, and A. Miranskyy, “On challenges of cloud monitoring,” in *Proc. 27th Annual Int. Conf. Comput. Sci. Softw. Eng.*, 2017, pp. 259–265.
- [7] A. Hrusto, E. Engström, and P. Runeson, “Towards optimization of anomaly detection in DevOps,” *Inf. Softw. Technol.*, vol. 160, Aug. 2023, Art. no. 107241.
- [8] P. Thantharate, “IntelligentMonitor: Empowering DevOps environments with advanced monitoring and observability,” in *Proc. Int. Conf. Inf. Technol. (ICIT)*, Aug. 2023, pp. 800–805.
- [9] A. Hrusto, “Enhancing DevOps with Autonomous Monitors: A proactive approach to failure detection,” Ph.D. thesis, Dept. Comput. Sci., Lund Univ., Lund, Sweden, Oct. 2024.
- [10] L. Ruff, J. R. Kauffmann, R. A. Vandermeulen, G. Montavon, W. Samek, M. Kloft, T. G. Dietterich, and K.-R. Müller, “A unifying review of deep and shallow anomaly detection,” *Proc. IEEE*, vol. 109, no. 5, pp. 756–795, May 2021.
- [11] R. Wu and E. J. Keogh, “Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress,” *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2421–2429, Mar. 2023.
- [12] S. Schmidl, P. Wenig, and T. Papenbrock, “Anomaly detection in time series: A comprehensive evaluation,” *Proc. VLDB Endowment*, vol. 15, no. 9, pp. 1779–1797, May 2022.
- [13] M. Braei and S. Wagner, “Anomaly detection in univariate time-series: A survey on the state-of-the-art,” 2020, *arXiv:2004.00433*.
- [14] V. Hodge and J. Austin, “A survey of outlier detection methodologies,” *Artif. Intell. Rev.*, vol. 22, no. 2, pp. 85–126, Oct. 2004.
- [15] N. Laptev, S. Amizadeh, and Y. Billawala, “S5—A labeled anomaly detection dataset, version 1.0 (16M),” Mar. 2015. [Online]. Available: <https://webscope.sandbox.yahoo.com/catalog.php?datatype=s&did=70>
- [16] S. Ahmad, A. Lavin, S. Purdy, and Z. Agha, “Unsupervised real-time anomaly detection for streaming data,” *Neurocomputing*, vol. 262, pp. 134–147, Nov. 2017.
- [17] P. Erik Strandberg and Y. Marklund, “The westermo test system performance data set,” 2023, *arXiv:2311.14510*.
- [18] M. Munir, S. A. Siddiqui, A. Dengel, and S. Ahmed, “DeepAnT: A deep learning approach for unsupervised anomaly detection in time series,” *IEEE Access*, vol. 7, pp. 1991–2005, 2019.
- [19] H. Xu, Y. Wang, S. Jian, Q. Liao, Y. Wang, and G. Pang, “Calibrated one-class classification for unsupervised time series anomaly detection,” *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 5723–5736, Nov. 2024.
- [20] S. Chauhan and S. Lee, “Machine learning-based anomaly detection for multivariate time series with correlation dependency,” *IEEE Access*, vol. 10, pp. 132062–132070, 2022.
- [21] K. Choi, J. Yi, C. Park, and S. Yoon, “Deep learning for anomaly detection in time-series data: Review, analysis, and guidelines,” *IEEE Access*, vol. 9, pp. 120043–120065, 2021.
- [22] Z. Li, N. Zhao, S. Zhang, Y. Sun, P. Chen, X. Wen, M. Ma, and D. Pei, “Constructing large-scale real-world benchmark datasets for AIOps,” 2022, *arXiv:2208.03938*.
- [23] G. Yu, P. Chen, Y. Li, H. Chen, X. Li, and Z. Zheng, “Nezha: Interpretable fine-grained root causes analysis for microservices on multi-modal observability data,” in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2023, pp. 553–565.

- [24] J. Ahmed and R. C. Green II, "Predicting severely imbalanced data disk drive failures with machine learning models," *Mach. Learn. Appl.*, vol. 9, Sep. 2022, Art. no. 100361.
- [25] D. Cotroneo, L. De Simone, P. Liguori, R. Natella, and N. Bidokhti, "How bad can a bug get? An empirical analysis of software failures in the OpenStack cloud computing platform," in *Proc. 27th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Aug. 2019, pp. 200–211.
- [26] J. Zhu, S. He, P. He, J. Liu, and M. R. Lyu, "Loghub: A large collection of system log datasets for AI-driven log analytics," in *Proc. IEEE 34th Int. Symp. Softw. Rel. Eng. (ISSRE)*, Oct. 2023, pp. 355–366.
- [27] A. Hrusto, E. Engström, and P. Runeson, "Optimization of anomaly detection in a microservice system through continuous feedback from development," in *Proc. IEEE/ACM 10th Int. Workshop Softw. Eng. Syst. Syst. Softw. Ecosyst. (SESoS)*, May 2022, pp. 13–20.
- [28] A. Hany Fawzy, K. Wassif, and H. Moussa, "Framework for automatic detection of anomalies in DevOps," *J. King Saud Univ.-Comput. Inf. Sci.*, vol. 35, no. 3, pp. 8–19, Mar. 2023.
- [29] M. S. Ali and D. Puri, "Optimizing DevOps methodologies with the integration of artificial intelligence," in *Proc. 3rd Int. Conf. Innov. Technol. (INOCON)*, Mar. 2024, pp. 1–5.
- [30] D. Sun, M. Fu, L. Zhu, G. Li, and Q. Lu, "Non-intrusive anomaly detection with streaming performance metrics and logs for DevOps in public clouds: A case study in AWS," *IEEE Trans. Emerg. Topics Comput.*, vol. 4, no. 2, pp. 278–289, Apr. 2016.
- [31] M. S. Floroiu, S. Russo, L. Giamattei, A. Guerriero, I. Malavolta, and R. Pietrantuono, "Anomaly detection and root cause analysis of microservices energy consumption," in *Proc. IEEE Int. Conf. Web Services (ICWS)*, Jul. 2024, pp. 590–600.
- [32] P. Wenig, S. Schmidl, and T. Papenbrock, "TimeEval: A benchmarking toolkit for time series anomaly detection algorithms," *Proc. VLDB Endowment*, vol. 15, no. 12, pp. 3678–3681, Aug. 2022.
- [33] E. Tarak and H. H. Kilinc, "DIA4M: A tool to streamline DevOps processes of distributed cloud-native systems," in *Proc. 9th Int. Conf. Comput. Sci. Eng. (UBMK)*, Oct. 2024, pp. 1110–1115.
- [34] N. Bosch and J. Bosch, "Software logs for machine learning in a DevOps environment," in *Proc. 46th Euromicro Conf. Softw. Eng. Adv. Appl. (SEAA)*, Aug. 2020, pp. 29–33.
- [35] I. Figalist, A. Biesdorf, C. Brand, S. Feld, and M. Kiermeier, "Supporting the DevOps feedback loop using unsupervised machine learning," in *Proc. IEEE Int. Symp. Innov. Intell. Syst. Appl. (INISTA)*, Jul. 2019, pp. 1–6.
- [36] M. S. Islam, W. Pourmajidi, L. Zhang, J. Steinbacher, T. Erwin, and A. Miranskyy, "Anomaly detection in a large-scale cloud platform," in *Proc. IEEE/ACM 43rd Int. Conf. Softw. Engineering: Softw. Eng. Pract. (ICSE-SEIP)*, May 2021, pp. 150–159.
- [37] G. Kånåhols, "Time series anomaly detection for DevOps test systems," Ph.D. Dissertation, School Innov., Des. Eng., Mälardalen Univ., Västerås, Sweden, 2024. [Online]. Available: <https://urn.kb.se/resolve?urn=urn:nbn:se:mdh:diva-67359>
- [38] C. C. Aggarwal, *Outlier Analysis*, 2nd ed., Cham, Switzerland: Springer, 2017.
- [39] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognit. Lett.*, vol. 27, no. 8, pp. 861–874, Jun. 2006.
- [40] K. Boyd, K. H. Eng, and C. D. Page, "Area under the precision-recall curve: Point estimates and confidence intervals," in *Machine Learning and Knowledge Discovery in Databases*, H. Blockeel, K. Kersting, S. Nijssen, and F. Železný, Eds., Berlin, Germany: Springer, 2013, pp. 451–466.
- [41] N. Tatbul, T. J. Lee, S. Zdonik, M. Alam, and J. Gottschlich, "Precision and recall for time series," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 31, Jan. 2018, pp. 1–12.
- [42] M. Thill, W. Konen, and T. Bäck, "Time series anomaly detection with discrete wavelet transforms and maximum likelihood estimation," in *Proc. Intern. Conf. Time Ser. (ITISE)*, vol. 2, 2017, pp. 11–23.
- [43] P. Senin, J. Lin, X. Wang, T. Oates, S. Gandhi, A. P. Boedihardjo, C. Chen, and S. Frankenstein, "Time series anomaly discovery with grammar-based compression," in *Proc. Edbt*, Jan. 2015, pp. 481–492.
- [44] T. Yairi, Y. Kato, and K. Hori, "Fault detection by mining association rules from house-keeping data," in *Proc. 6th Int. Symp. Artif. Intell., Robot. Autom. Space*, vol. 18, 2001, p. 21.
- [45] P. Malhotra, L. Vig, G. Shroff, and P. Agarwal, "Long short term memory networks for anomaly detection in time series," in *Proc. Esann*, Jan. 2015, p. 89.
- [46] F. T. Liu, K. M. Ting, and Z. Zhou, "Isolation forest," in *Proc. 8th IEEE Int. Conf. Data Mining*, Dec. 2008, pp. 413–422.
- [47] M. M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, "LOF: Identifying density-based local outliers," *ACM SIGMOD Rec.*, vol. 29, no. 2, pp. 93–104, May 2000.
- [48] C. R. Harris et al., "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, Sep. 2020.
- [49] S. Geman, E. Bienenstock, and R. Doursat, "Neural networks and the bias/variance dilemma," *Neural Comput.*, vol. 4, no. 1, pp. 1–58, Jan. 1992.
- [50] T. G. Dietterich, "Ensemble methods in machine learning," in *Proc. Int. Workshop Multiple Classifier Syst.* Cham, Switzerland: Springer, Jan. 2000, pp. 1–15.



GUSTAV KÅNÅHOLS received the bachelor's degree in computer sciences from Mälardalen University, in 2024. During his time at the university, he was the President of the Association of Computer Science and Network Technology Students, for two years. He is currently a Software Developer and a Consultant in Västerås.



SHAHRIAR HASAN received the joint master's degree in internet technologies and architecture from Sorbonne Université, Paris, France, and the University of Trento, Italy, in 2017, and the Ph.D. degree in computer science and engineering from Mälardalen University, Sweden, in 2023. From 2013 to 2015, he was a Software Engineer with the Samsung Research and Development Institute Bangladesh Ltd., Dhaka, Bangladesh. In 2017, he completed an internship with the Fondazione Bruno Kessler (FBK), Trento, Italy. He was a Lecturer in computer science with American International University, Bangladesh, from 2017 to 2018. He is currently a Lecturer in data communications and embedded systems with Mälardalen University. He has contributed to several European projects focusing on 5G radio access networks, vehicle platooning, and AI for cyber-physical systems. His research interests include connected and automated vehicles, wireless communications, artificial intelligence, teleoperation, and cyber-physical systems-of-systems.



PER ERIK STRANDBERG received the first master's degree in applied mathematics and the second master's degree in bioinformatics from Linköping University, Sweden, in 2004 and 2005, respectively and the Industrial Ph.D. degree in test automation from Mälardalen University, in 2021. After working a more than a decade in the IT domain with software engineering and in particular software testing, he started as an industrial doctoral student on the topic of test automation with Westermo Network Technologies AB and Mälardalen University, in 2017, Västerås, Sweden. After received the Ph.D. degree, he started working as the Project Manager with Westermo Network Technologies AB, in research projects with industry-academia co-production. He is active in research on software engineering, requirements engineering, software test automation, applied artificial intelligence, and data analytics.

...